

CourtrightCollective

START AT HOUR FORTY

COURTRIGHT COLLECTIVE

The Indie App Starter *Kit*

A real, working app scaffold. No framework, no build step, no npm install. Clone it, rename it, ship it this weekend.

ZERO DEPENDENCIES

PWA + OFFLINE

DEPLOYS FREE

COURTRIGHTCO.COM

BUILT IN NASHVILLE

The Indie App Starter Kit

What this is

A complete, working app you can open in a browser right now. Router, state store with persistence, dark mode, onboarding, empty states, a confirm modal, toasts, an offline banner, a service worker, a PWA manifest, icons, a serverless API endpoint, and deploy config.

Zero dependencies. Zero build step. No `npm install`, no bundler, no framework version to keep up with. Open `starter/index.html` through a local server and it runs.

The point is not that you should never use a framework. The point is that the first forty hours of every project get burned on the same setup work, and those forty hours are exactly where solo projects die. This is hour forty, already done.

Run it in 30 seconds

You need a local server. Opening `index.html` by double-clicking uses the `file://` protocol, which blocks service workers and ES modules.

```
cd starter

# Any one of these works
python3 -m http.server 8000
npx serve .
php -S localhost:8000
```

Then open `http://localhost:8000`.

What is in the box

```
starter/
  index.html      App shell, meta tags, onboarding, modal, toast
  app.css         Every style. Tokens at the top, rest follows
  app.js          Store, router, views, UI helpers. ~400 lines
  sw.js           Service worker. Precached shell, network-first pages
  manifest.webmanifest PWA install config
  _headers        Cloudflare Pages security and cache headers
  _redirects       Cloudflare Pages redirects
  .gitignore
  icons/
    icon-192.png   Placeholder app icon
    icon-512.png   Placeholder app icon
    icon-maskable-512.png Android adaptive icon, art inside the safe zone
    og.png         1200x630 social preview
  functions/
    api/track.js   A working Cloudflare Pages Function

docs/
  01-ARCHITECTURE.md How the pieces fit, and why each choice was made
  02-CUSTOMIZE.md    Make it yours in about an hour
  03-DEPLOY.md       Live on a custom domain, free, in 10 minutes
  04-LAUNCH-CHECKLIST.md The pre-launch list, 60 items
```

What it already does

FEATURE	WHERE IT LIVES	WHY IT IS HERE
Hash router	<code>app.js</code> section 7	Works on any static host with zero config
Persistent store	<code>app.js</code> section 2	localStorage with merge-on-load and a migration hook
Dark mode	<code>app.css</code> tokens + <code>app.js</code> section 5	Auto, light, dark. Only the tokens change
Onboarding	<code>index.html</code> + <code>app.js</code> section 8	Shows once, ever
Empty states	<code>app.js</code> <code>/list</code> view	The most-seen screen in any new app
Confirm modal	<code>app.js</code> section 4	Promise-based, escape closes it, backdrop closes it
Toasts	<code>app.js</code> section 4	Feedback without a dialog
Offline banner	<code>index.html</code> + <code>app.js</code> section 8	Honest about connectivity
Service worker	<code>sw.js</code>	Opens with no network at all
Feature flags	<code>app.js</code> <code>CONFIG.features</code>	Ship things off rather than deleting them
Analytics stub	<code>app.js</code> section 3	Three events, wired to a real endpoint when ready
Data export	<code>/settings</code> view	Users owning their data is table stakes
Reset	<code>/settings</code> view	And it actually works
Serverless endpoint	<code>functions/api/track.js</code>	A real Pages Function with validation
Accessibility basics	Throughout	Visible focus, aria-current, reduced motion respected

The design decisions, stated plainly

Why no framework. Not ideology. A solo v1 rarely has state complex enough to need one, and a framework adds a build step, a dependency tree, and a version treadmill. When your app outgrows this, you will know, and porting a working app is far easier than finishing a half-built one.

Why hash routing. `#/list` instead of `/list`. It works on every static host with no rewrite rules, it works when you open the app from a subdirectory, and it never 404s on refresh. Switch to the History API when you need real URLs for SEO.

Why localStorage. Synchronous, universally supported, and enough for a v1's worth of data. It is capped around 5MB and it is per-origin. When you outgrow it, move to IndexedDB and keep the same `Store` interface, so nothing above it changes.

Why zero images in the UI. Every image is a request that can fail, a file to optimize, and a thing to redo when you rebrand. The app icon, the empty state mark, and the onboarding badge are all CSS.

Why the CSS is one file. It is under 500 lines. Splitting it into eight files would be tidier and slower to work in. Split it when it hurts.

Why fonts are system fallbacks. The app names Outfit and Cormorant Garamond and falls back to system fonts, so it makes zero external requests out of the box. Add real fonts by self-hosting them and adding `@font-face` at the top of `app.css`. Never link a font CDN in an app you want to work offline.

Your first hour

1. **Rename it.** Search the whole project for `Starter` and replace with your app name. Six files.
2. **Change three colors** in `app.css`: `--accent`, `--accent-2`, `--bg`. That is your entire brand.
3. **Replace the icons** in `icons/`. The placeholders are real PNGs at the right sizes, so you can ship without touching them, but you should touch them.
4. **Change** `CONFIG.storageKey` in `app.js` so your app does not share storage with the starter.
5. **Delete the `/list` view and build your thing.** That view is a complete feature in miniature: form, store update, render, event delegation, confirm-before-destroy, persistence. Copy the shape, change the content.
6. **Deploy it.** See `docs/03-DEPLOY.md`. Free, ten minutes, custom domain included.

Ship something ugly on day one and improve it in public. That beats a perfect app nobody has seen.

What this deliberately does not include

Being honest about this up front so you are not hunting for something that is not there:

- **No auth.** Most v1 apps do not need accounts, and adding them before you have users is how you spend three weeks on a login screen nobody has used yet.
- **No database.** Local storage covers a v1. The Pages Function shows where a real backend would attach.
- **No payments.** Wire Stripe Checkout or the platform's in-app purchase when you have something worth charging for.
- **No tests.** For a single-file v1, manual testing on a real device is faster and catches more. Add tests when you have logic worth protecting.
- **No TypeScript.** Add it if you like it. It is a build step, which is the one thing this kit is deliberately without.

Every one of those is a deliberate omission, not a gap. Adding them before you need them is the most common way an indie project runs out of momentum.

License

Yours. Use it for client work, use it for your own apps, ship it commercially, modify it however you want. No attribution required.

Do not resell the kit itself as a starter kit. Everything you build on top of it is entirely yours.

courtrightco.com